

ML Implementations

Bryan Martinez

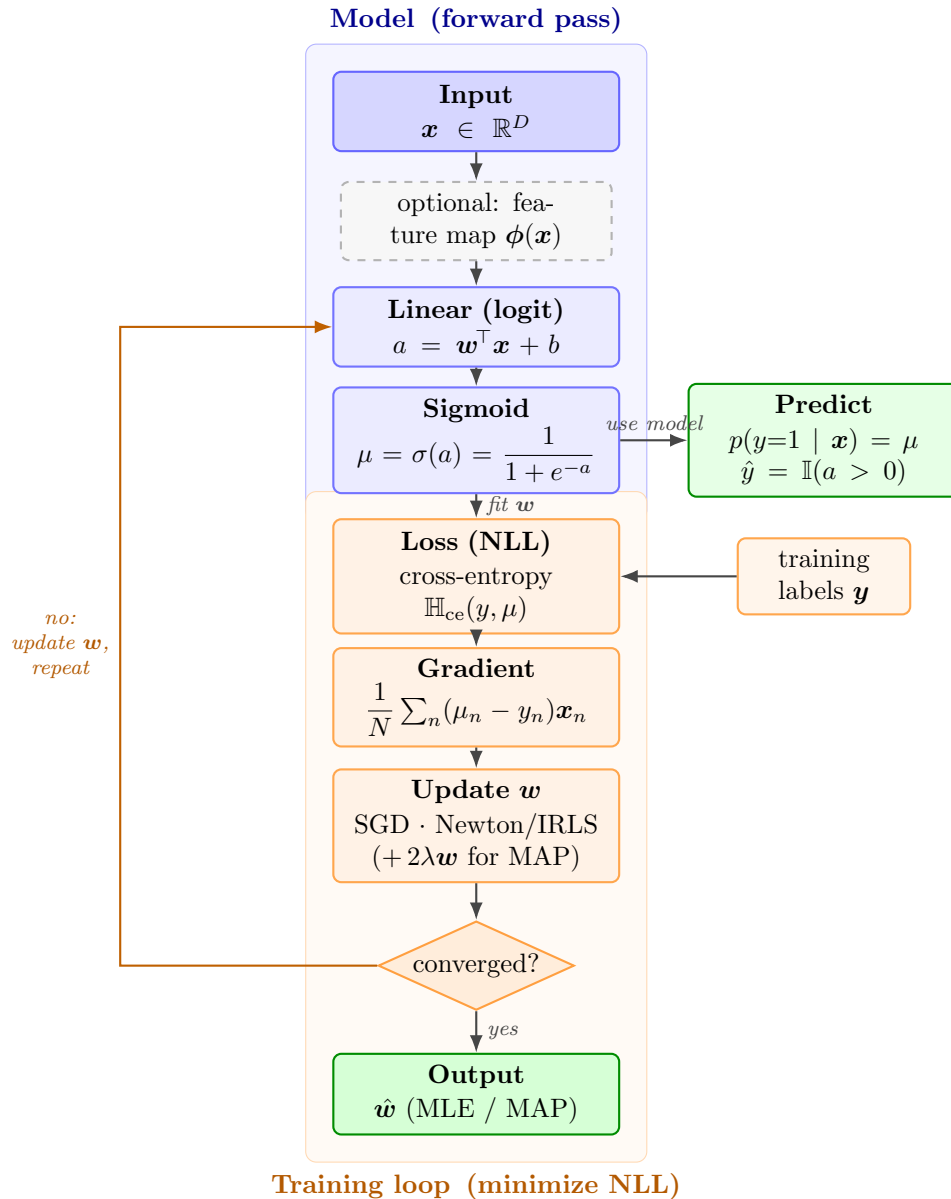
June 1, 2026

Contents

1	Logistic Regression	3
1.1	Binary Logistic Regression	4
1.1.1	Linear Classifiers	4
1.1.2	Nonlinear Classifiers	5
1.2	Maximum Likelihood Estimation	6
1.2.1	Objective Function	6
1.2.2	Optimizing The Objective	8
1.3	Stochastic Gradient Descent	9
1.3.1	Iteratively Reweighted Least Squares	9
1.3.2	MAP Estimation	11
1.3.3	Standardization	12
1.4	Robust Logistic Regression	12
1.4.1	Mixture Model for the likelihood	12
1.4.2	Bi-tempered Loss	13
1.5	Bayesian Logistic Regression	14
1.5.1	Laplace Approximation	14
1.5.2	Posterior Predictive Distribution	15
2	Linear Regression	16
2.1	Least Square Linear Regression	17
2.1.1	Terminology	17
2.2	Least Squares Linear Regression	18
2.2.1	Objective Function	18
2.2.2	Ordinary Least Squares	19
2.2.3	Geometric Interpretation	19
2.2.4	Solving the Normal Equations Numerically	20
2.3	Weighted Least Squares	21
2.3.1	Other Approaches to Computing the MLE	21
2.4	Measuring Goodness of Fit	23
2.4.1	Residual Plots	23
2.4.2	Prediction Accuracy and R^2	23
2.5	Ridge Regression	24
2.5.1	Computing the MAP Estimate	25
2.5.2	Connection Between Ridge Regression and PCA	26
2.5.3	Choosing the Strength of the Regularizer	27

2.6	Lasso Regression	27
2.6.1	MAP Estimation with a Laplace Prior (ℓ_1 Regularization)	28
2.6.2	Why Does ℓ_1 Regularization Yield Sparse Solutions?	29
2.6.3	Elastic Net (Ridge and Lasso Combined)	29
2.6.4	Optimization Algorithms	30

1 Logistic Regression



Logistic regression is a discriminative classification model of the form $p(y \mid \mathbf{x}; \boldsymbol{\theta})$, modeling the label distribution directly without modeling $p(\mathbf{x})$.

- $\mathbf{x} \in \mathbb{R}^D$: fixed-dimensional input (feature) vector.
- $y \in \{1, \dots, C\}$: class label.
- $\boldsymbol{\theta}$: model parameters.

Two cases, depending on the number of classes C :

- $C = 2$: **binary logistic regression**.
- $C > 2$: **multinomial** (equivalently, **multiclass**) logistic regression. (need to add to notes)

later)

1.1 Binary Logistic Regression

Binary logistic regression corresponds to the model

$$p(y \mid \mathbf{x}, \boldsymbol{\theta}) = \text{Ber}(y \mid \sigma(\mathbf{w}^\top \mathbf{x} + b)),$$

where σ is the sigmoid function, $\mathbf{w}$ are the weights, b is the bias, and $\boldsymbol{\theta} = (\mathbf{w}, b)$ collects all the parameters. Equivalently,

$$p(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}) = \sigma(a) = \frac{1}{1 + e^{-a}},$$

where

$$a = \mathbf{w}^\top \mathbf{x} + b = \log\left(\frac{p}{1-p}\right)$$

is the **log-odds**, with $p = p(y = 1 \mid \mathbf{x}, \boldsymbol{\theta})$. In ML, a is usually called the **logit** or the **pre-activation**.

Alternative label convention. Sometimes labels $\tilde{y} \in \{-1, +1\}$ are used in place of $y \in \{0, 1\}$. Because $\sigma(-a) = 1 - \sigma(a)$, the probability of either alternative label can be written compactly as

$$p(\tilde{y} \mid \mathbf{x}, \boldsymbol{\theta}) = \sigma(\tilde{y} a).$$

This more compact notation is widely used in the ML literature.

1.1.1 Linear Classifiers

Optimal decision rule. The sigmoid gives $p(y = 1 \mid \mathbf{x})$. If misclassifying either class incurs the same loss, the optimal rule is to predict $y = 1$ exactly when class 1 is more likely than class 0:

$$f(\mathbf{x}) = \mathbb{I}(p(y = 1 \mid \mathbf{x}) > p(y = 0 \mid \mathbf{x})) = \mathbb{I}\left(\log \frac{p(y = 1 \mid \mathbf{x})}{p(y = 0 \mid \mathbf{x})} > 0\right) = \mathbb{I}(a > 0),$$

where $a = \mathbf{w}^\top \mathbf{x} + b$.

Prediction function. The decision is driven by

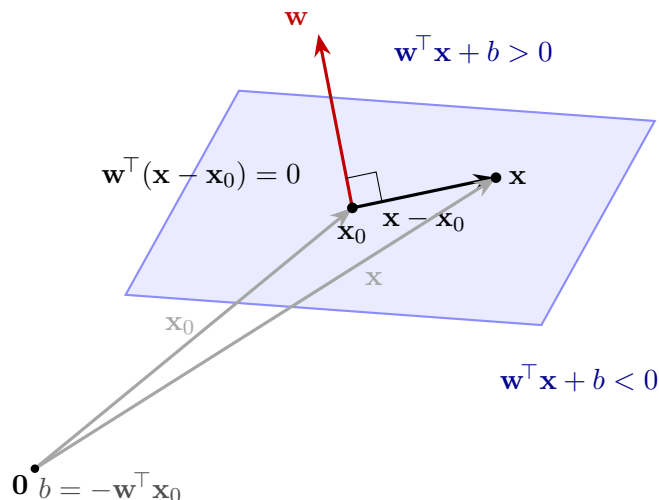
$$f(\mathbf{x}; \boldsymbol{\theta}) = b + \mathbf{w}^\top \mathbf{x} = b + \sum_{d=1}^D w_d x_d,$$

where $\mathbf{w}^\top \mathbf{x} = \langle \mathbf{w}, \mathbf{x} \rangle$ is the inner product of the weight vector $\mathbf{w}$ and feature vector $\mathbf{x}$. This defines a linear **hyperplane** with normal vector $\mathbf{w} \in \mathbb{R}^D$ and offset $b \in \mathbb{R}$ from the origin.

Decision boundary geometry. Consider a plane through a point $\mathbf{x}_0$ with surface normal $\mathbf{w}$. Points on the surface satisfy $\mathbf{w}^\top (\mathbf{x} - \mathbf{x}_0) = 0$. Defining $b = -\mathbf{w}^\top \mathbf{x}_0$ rewrites this as

$$\mathbf{w}^\top \mathbf{x} + b = 0,$$

which splits the space into two half-spaces. This linear surface is the **decision boundary**.



Linear separability. If a linear boundary can perfectly separate the training examples (zero training errors), the data is **linearly separable**. The 2-class, 2-feature version of the iris dataset is *not* linearly separable.

Predicting a distribution. In general there is uncertainty about the true label, so we predict a full distribution over labels rather than only deciding which side of the boundary a point lies on. For $p(y = 1 \mid (x_1, x_2), \mathbf{w}) = \sigma(w_1 x_1 + w_2 x_2)$:

- The *direction* of $\mathbf{w}$ sets the orientation of the decision boundary.
- The *magnitude* $\|\mathbf{w}\| = \sqrt{\sum_{d=1}^D w_d^2}$ controls the steepness of the sigmoid, and hence the confidence of the predictions.

1.1.2 Nonlinear Classifiers

Feature transformations for separability. A problem that is not linearly separable in the raw inputs can often be made so by preprocessing the features. Let $\phi(\mathbf{x})$ denote a transformed version of the input vector; we then fit a linear model in $\phi(\mathbf{x})$ rather than in $\mathbf{x}$.

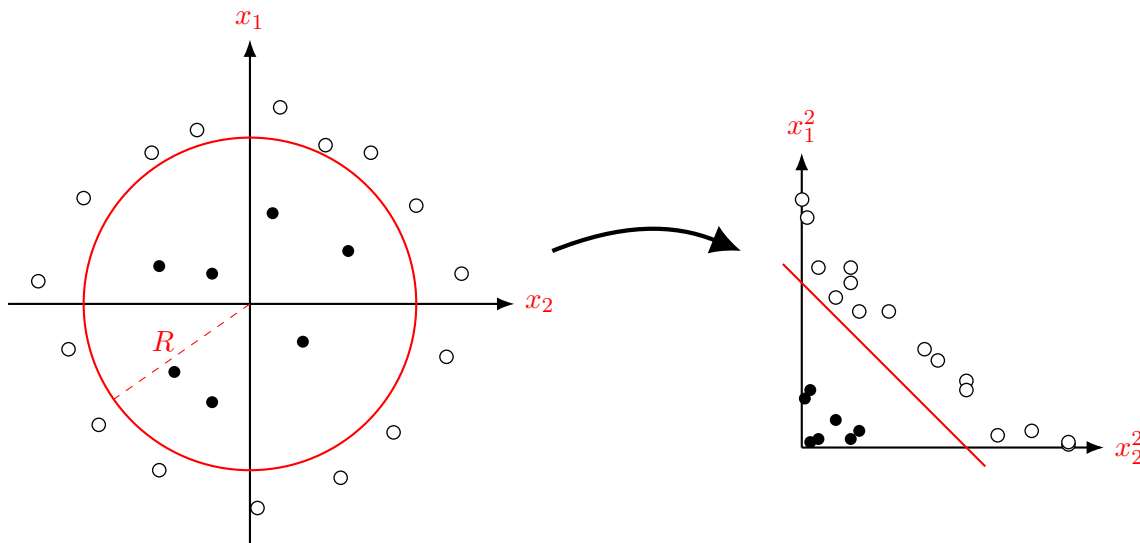
Example (quadratic $\rightarrow$ circular boundary). Take

$$\phi(x_1, x_2) = [1, x_1^2, x_2^2], \quad \mathbf{w} = [-R^2, 1, 1].$$

Then

$$\mathbf{w}^\top \phi(\mathbf{x}) = x_1^2 + x_2^2 - R^2,$$

so the decision boundary $f(\mathbf{x}) = 0$ is the circle $x_1^2 + x_2^2 = R^2$ of radius R . Mapping $\mathbf{x} = (x_1, x_2) \mapsto (x_1^2, x_2^2)$ turns this quadratic boundary into a *linear* one in the transformed space.



Key point. Even after the transformation, f is still *linear in the parameters* $\mathbf{w}$, which keeps the learning problem simple. The nonlinearity lives entirely in the fixed feature map ϕ .

Learning the features. More power is available by *learning* the parameters of the feature extractor $\phi(\mathbf{x})$ jointly with the linear weights $\mathbf{w}$, rather than fixing ϕ by hand (discussed in Part III).

Polynomial expansion and overfitting. The quadratic map generalizes to higher-order polynomial expansions up to some degree K . As K (and hence the number of parameters) grows, the model becomes more flexible, but eventually *overfits*.

1.2 Maximum Likelihood Estimation

Estimating the parameters of a logistic regression model using maximum likelihood estimation (MLE)

1.2.1 Objective Function

The negative log likelihood (scaled by the dataset size N) is given by the following: **Note:** We assume the bias term b is absorbed into the weight vector $\mathbf{w}$:

Negative log-likelihood (binary cross-entropy). Using labels $y_n \in \{0, 1\}$, the average NLL of the data $\mathcal{D}$ under the model is

$$\text{NLL}(\mathbf{w}) = -\frac{1}{N} \log p(\mathcal{D} | \mathbf{w}) = -\frac{1}{N} \log \prod_{n=1}^N \text{Ber}(y_n | \mu_n) \quad (1)$$

$$= -\frac{1}{N} \sum_{n=1}^N \log [\mu_n^{y_n} \times (1 - \mu_n)^{1-y_n}] \quad (2)$$

$$= -\frac{1}{N} \sum_{n=1}^N [y_n \log \mu_n + (1 - y_n) \log(1 - \mu_n)] \quad (3)$$

$$= \frac{1}{N} \sum_{n=1}^N \mathbb{H}_{\text{ce}}(y_n, \mu_n), \quad (4)$$

where $\mu_n = \sigma(a_n)$ is the predicted probability of class 1, $a_n = \mathbf{w}^\top \mathbf{x}_n$ is the logit, and $\mathbb{H}_{\text{ce}}$ is the **binary cross entropy**

$$\mathbb{H}_{\text{ce}}(p, q) = -[p \log q + (1 - p) \log(1 - q)].$$

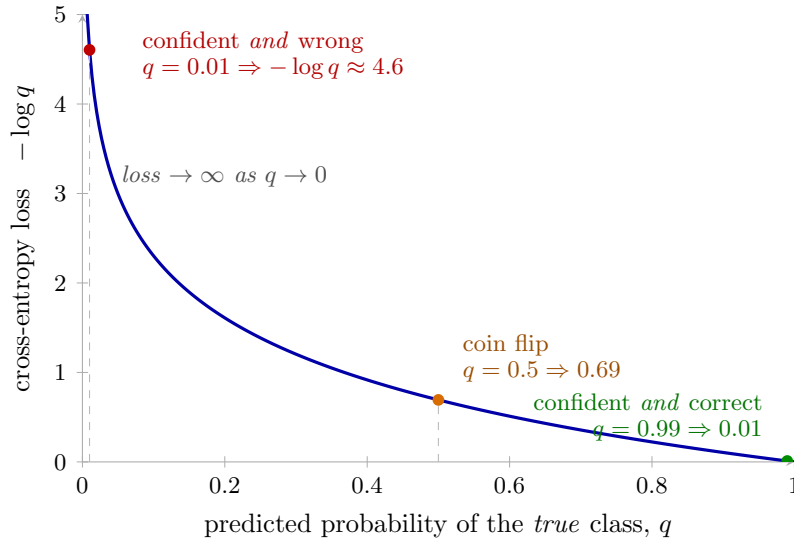
Cross entropy in general. For two distributions p and q over C classes, the cross entropy is

$$\mathbb{H}(p, q) = - \sum_{c=1}^C p_c \log q_c.$$

The binary loss above is the $C = 2$ case with true distribution $p = (y, 1 - y)$ and predicted $q = (\mu, 1 - \mu)$. It decomposes as

$$\mathbb{H}(p, q) = \underbrace{\mathbb{H}(p)}_{\text{entropy of } p} + \underbrace{D_{\text{KL}}(p \parallel q)}_{\geq 0},$$

so minimizing cross entropy over q is the same as minimizing the KL divergence $D_{\text{KL}}(p \parallel q)$ — i.e. pushing the predicted distribution q as close as possible to the true distribution p . Because the empirical label distribution p is fixed, $\mathbb{H}(p)$ is a constant, so *minimizing the NLL, minimizing cross entropy, and minimizing KL divergence all coincide.*



± 1 label form. With $\tilde{y}_n \in \{-1, +1\}$ instead of $y_n \in \{0, 1\}$, the same loss can be rewritten using $\sigma(-a) = 1 - \sigma(a)$:

$$\text{NLL}(\mathbf{w}) = -\frac{1}{N} \sum_{n=1}^N [\mathbb{I}(\tilde{y}_n = 1) \log \sigma(a_n) + \mathbb{I}(\tilde{y}_n = -1) \log \sigma(-a_n)] \quad (5)$$

$$= -\frac{1}{N} \sum_{n=1}^N \log \sigma(\tilde{y}_n a_n) \quad (6)$$

$$= \frac{1}{N} \sum_{n=1}^N \log(1 + \exp(-\tilde{y}_n a_n)). \quad (7)$$

Convention. We mostly use $y_n \in \{0, 1\}$.

1.2.2 Optimizing The Objective

To find the MLE we need to solve:

$$\nabla_{\mathbf{w}} \text{NLL}(\mathbf{w}) = \mathbf{g}(\mathbf{w}) = \mathbf{0}$$

This can be solved using any gradient-based optimization algorithm.

Gradient of the NLL. Start from the sigmoid derivative

$$\frac{d\mu_n}{da_n} = \sigma(a_n)(1 - \sigma(a_n)), \quad a_n = \mathbf{w}^\top \mathbf{x}_n, \quad \mu_n = \sigma(a_n).$$

By the chain rule,

$$\frac{\partial}{\partial w_d} \mu_n = \frac{\partial}{\partial w_d} \sigma(\mathbf{w}^\top \mathbf{x}_n) = \frac{\partial \sigma(a_n)}{\partial a_n} \frac{\partial a_n}{\partial w_d} = \mu_n(1 - \mu_n) x_{nd}.$$

(The bias gradient follows identically by setting $x_{n0} = 1$; we drop the bias for simplicity.) Hence

$$\begin{aligned} \nabla_{\mathbf{w}} \log \mu_n &= \frac{1}{\mu_n} \nabla_{\mathbf{w}} \mu_n = (1 - \mu_n) \mathbf{x}_n, \\ \nabla_{\mathbf{w}} \log(1 - \mu_n) &= \frac{-\mu_n(1 - \mu_n) \mathbf{x}_n}{1 - \mu_n} = -\mu_n \mathbf{x}_n. \end{aligned}$$

Substituting into the NLL gradient and simplifying:

$$\nabla_{\mathbf{w}} \text{NLL}(\mathbf{w}) = -\frac{1}{N} \sum_{n=1}^N [y_n(1 - \mu_n) \mathbf{x}_n - (1 - y_n)\mu_n \mathbf{x}_n] \quad (8)$$

$$= -\frac{1}{N} \sum_{n=1}^N [y_n \mathbf{x}_n - y_n \mu_n \mathbf{x}_n - \mu_n \mathbf{x}_n + y_n \mu_n \mathbf{x}_n] \quad (9)$$

$$= \frac{1}{N} \sum_{n=1}^N (\mu_n - y_n) \mathbf{x}_n. \quad (10)$$

Interpretation. Writing the **error signal** $e_n = \mu_n - y_n$, the gradient weights each input $\mathbf{x}_n$ by its error and averages over the data.

Matrix form.

$$\nabla_{\mathbf{w}} \text{NLL}(\mathbf{w}) = \frac{1}{N} \left(\mathbf{1}_N^\top (\text{diag}(\boldsymbol{\mu} - \mathbf{y}) \mathbf{X}) \right)^\top,$$

where $\mathbf{X}$ is the $N \times D$ design matrix with each example $\mathbf{x}_n$ as a row.

Convexity via the Hessian. A gradient-based optimizer finds a stationary point where $\mathbf{g}(\mathbf{w}) = \mathbf{0}$, which may be a global or merely local optimum. To guarantee it is the *global* optimum, it suffices to show the objective is **convex** — intuitively, that the NLL is bowl-shaped with a unique minimum (Fig. 10.5b). Formally, we show the Hessian is positive semi-definite.

The Hessian of the NLL is

$$\mathbf{H}(\mathbf{w}) = \nabla_{\mathbf{w}} \nabla_{\mathbf{w}}^{\top} \text{NLL}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \mu_n(1 - \mu_n) \mathbf{x}_n \mathbf{x}_n^{\top} = \frac{1}{N} \mathbf{X}^{\top} \mathbf{S} \mathbf{X},$$

where

$$\mathbf{S} \triangleq \text{diag}(\mu_1(1 - \mu_1), \dots, \mu_N(1 - \mu_N)).$$

Positive definiteness. For any nonzero vector $\mathbf{v}$,

$$\mathbf{v}^{\top} \mathbf{X}^{\top} \mathbf{S} \mathbf{X} \mathbf{v} = (\mathbf{v}^{\top} \mathbf{X}^{\top} \mathbf{S}^{1/2}) (\mathbf{S}^{1/2} \mathbf{X} \mathbf{v}) = \|\mathbf{S}^{1/2} \mathbf{X} \mathbf{v}\|_2^2 > 0,$$

which holds because $\mu_n > 0$ for all n (a consequence of the sigmoid). Hence $\mathbf{H}$ is positive definite and the NLL is **strictly convex**.

Practical caveat. When some μ_n are close to 0 or 1, the corresponding entries of $\mathbf{S}$ shrink toward zero and the Hessian can become near-singular.

1.3 Stochastic Gradient Descent

Our goal is to solve the following optimization problem:

$$\hat{\mathbf{w}} \triangleq \underset{\mathbf{w}}{\text{argmin}} \mathcal{L}(\mathbf{w}),$$

where the loss $\mathcal{L}(\mathbf{w})$ is the negative log-likelihood

$$\text{NLL}(\mathbf{w}) = -\frac{1}{N} \sum_{n=1}^N [y_n \log \mu_n + (1 - y_n) \log(1 - \mu_n)],$$

with $\mu_n = \sigma(a_n)$ the probability of class 1 and $a_n = \mathbf{w}^{\top} \mathbf{x}_n$ the log-odds. There is no closed-form solution, so we minimize numerically.

Stochastic gradient descent. The simplest method is SGD. With a minibatch of size 1, the update is

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \nabla_{\mathbf{w}} \text{NLL}(\mathbf{w}_t) = \mathbf{w}_t - \eta_t (\mu_n - y_n) \mathbf{x}_n,$$

where the full average over all N examples in the gradient is replaced by a single stochastically chosen sample n (and the index n changes with t).

Convergence. Since the objective is convex, this procedure converges to the *global* optimum provided the learning rate η_t is decayed at a suitable rate (Section 8.4.3). Convergence speed can be improved with variance-reduction techniques such as SAGA (Section 8.4.5.2).

1.3.1 Iteratively Reweighted Least Squares

Second-order optimization. Gradient descent is a *first-order* method — it uses only the gradient. This is slow on ill-conditioned problems, where some directions are steep and others shallow. A *second-order* method exploits the curvature (the Hessian) and can converge far faster. We work in the **full-batch** setting (N small), since second-order methods are harder to make work stochastically.

Newton's method. The classic second-order update is

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \mathbf{H}_t^{-1} \mathbf{g}_t, \quad \mathbf{H}_t \triangleq \nabla^2 \mathcal{L}(\mathbf{w})|_{\mathbf{w}_t} = \mathbf{H}(\mathbf{w}_t),$$

where $\mathbf{H}_t$ is assumed positive-definite (so the step is well-defined). With the exact Hessian one can take $\eta_t = 1$.

Newton update for logistic regression. Recall the gradient and Hessian

$$\mathbf{g} = \nabla_{\mathbf{w}} \text{NLL}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N (\mu_n - y_n) \mathbf{x}_n, \quad \mathbf{H} = \frac{1}{N} \mathbf{X}^\top \mathbf{S} \mathbf{X},$$

$$\mathbf{S} \triangleq \text{diag}(\mu_1(1 - \mu_1), \dots, \mu_N(1 - \mu_N)).$$

Substituting (with $\eta_t = 1$) and simplifying:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \mathbf{H}^{-1} \mathbf{g}_t \tag{11}$$

$$= \mathbf{w}_t + (\mathbf{X}^\top \mathbf{S}_t \mathbf{X})^{-1} \mathbf{X}^\top (\mathbf{y} - \boldsymbol{\mu}_t) \tag{12}$$

$$= (\mathbf{X}^\top \mathbf{S}_t \mathbf{X})^{-1} [(\mathbf{X}^\top \mathbf{S}_t \mathbf{X}) \mathbf{w}_t + \mathbf{X}^\top (\mathbf{y} - \boldsymbol{\mu}_t)] \tag{13}$$

$$= (\mathbf{X}^\top \mathbf{S}_t \mathbf{X})^{-1} \mathbf{X}^\top [\mathbf{S}_t \mathbf{X} \mathbf{w}_t + \mathbf{y} - \boldsymbol{\mu}_t] \tag{14}$$

$$= (\mathbf{X}^\top \mathbf{S}_t \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{S}_t \mathbf{z}_t, \tag{15}$$

where the **working response** is

$$\mathbf{z}_t \triangleq \mathbf{X} \mathbf{w}_t + \mathbf{S}_t^{-1} (\mathbf{y} - \boldsymbol{\mu}_t), \quad \mathbf{S}_t = \text{diag}(\mu_{t,n}(1 - \mu_{t,n})).$$

Since $\mathbf{S}_t$ is diagonal, componentwise

$$z_{t,n} = \mathbf{w}_t^\top \mathbf{x}_n + \frac{y_n - \mu_{t,n}}{\mu_{t,n}(1 - \mu_{t,n})}.$$

Weighted least squares. The last line is a **weighted least squares** problem: it minimizes

$$\sum_{n=1}^N S_{t,n} (z_{t,n} - \mathbf{w}_t^\top \mathbf{x}_n)^2.$$

Because the weights $\mathbf{S}_t$ are recomputed each iteration, the method is called **iteratively reweighted least squares (IRLS)**.

Fisher scoring. A variant that replaces the actual Hessian with its expectation — the *Fisher information matrix* — rather than $\mathbf{H}$. Since the Fisher information is data-independent, it can be precomputed (unlike $\mathbf{H}$, which must be re-evaluated every iteration), which can be faster when there are many parameters.

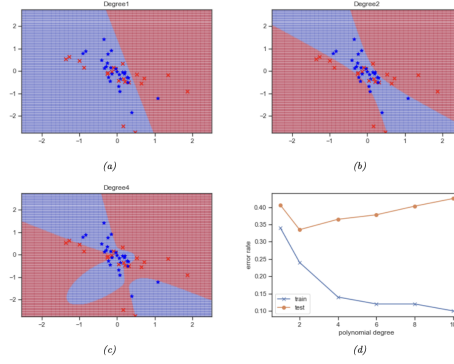


Figure 10.4: Polynomial feature expansion applied to a two-class, two-dimensional logistic regression problem. (a) Degree $K = 1$. (b) Degree $K = 2$. (c) Degree $K = 4$. (d) Train and test error vs degree. Generated by `logreg_poly_demo.sphinx`.

1.3.2 MAP Estimation

Why overfitting happens. Logistic regression overfits when there are too many parameters relative to training examples (Fig. 10.4). Maximum likelihood finds weights that make the decision boundary “wiggle” to curve tightly around the training points, and achieving this typically requires *large-magnitude* weights. For example, fitting polynomial expansions of increasing degree K to the data in Fig. 10.4 gives MLE weights that grow dramatically with K :

$$K = 1 : \quad \hat{\mathbf{w}} = [0.513, 0.119], \quad (16)$$

$$K = 2 : \quad \hat{\mathbf{w}} = [2.275, 0.060, 11.842, 15.404, 2.512], \quad (17)$$

$$K = 4 : \quad \hat{\mathbf{w}} = [-3.078, \dots, -59.032, 51.772, 10.251]. \quad (18)$$

ℓ_2 regularization (weight decay). To curb overfitting, prevent the weights from growing large. Place a zero-mean Gaussian prior $p(\mathbf{w}) = \mathcal{N}(\mathbf{w} \mid \mathbf{0}, C\mathbf{I})$ and use MAP estimation. The training objective becomes

$$\mathcal{L}(\mathbf{w}) = \text{NLL}(\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2, \quad \|\mathbf{w}\|_2^2 = \sum_{d=1}^D w_d^2, \quad \lambda = \frac{1}{C}.$$

This is **ℓ_2 regularization** (a.k.a. weight decay). A larger λ penalizes large weights more heavily (forcing them toward the zero-mean prior), yielding a less flexible model (Fig. 10.6). Equivalently, C is the prior variance, so larger $C \Leftrightarrow$ weaker regularization.

MAP via the same optimizers. The MAP estimate just requires small modifications to the gradient and Hessian. With the penalized NLL

$$\text{PNLL}(\mathbf{w}) = \text{NLL}(\mathbf{w}) + \lambda \mathbf{w}^\top \mathbf{w},$$

we have

$$\nabla_{\mathbf{w}} \text{PNLL}(\mathbf{w}) = \mathbf{g}(\mathbf{w}) + 2\lambda \mathbf{w}, \quad \nabla_{\mathbf{w}}^2 \text{PNLL}(\mathbf{w}) = \mathbf{H}(\mathbf{w}) + 2\lambda \mathbf{I},$$

where $\mathbf{g}(\mathbf{w})$ and $\mathbf{H}(\mathbf{w})$ are the gradient and Hessian of the *unpenalized* NLL. (For a related exercise, see Exercise 10.2.)

1.3.3 Standardization

The isotropic prior $\mathcal{N}(\mathbf{w} \mid \mathbf{0}, \lambda^{-1} \mathbf{I})$ used for ℓ_2 regularization penalizes every weight equally. This implicitly assumes the weights should be similar in magnitude — which in turn assumes the *input features* are on similar scales. In practice features are often on wildly different scales (e.g. a length in cm vs. a count), so this assumption fails.

Standardization fixes this by rescaling each feature to have mean 0 and variance 1:

$$\text{standardize}(x_{nd}) = \frac{x_{nd} - \hat{\mu}_d}{\hat{\sigma}_d}, \quad \hat{\mu}_d = \frac{1}{N} \sum_{n=1}^N x_{nd}, \quad \hat{\sigma}_d^2 = \frac{1}{N} \sum_{n=1}^N (x_{nd} - \hat{\mu}_d)^2.$$

An alternative is **min-max scaling**, which rescales each feature to lie in $[0, 1]$.

Why it helps. Both methods put the features on comparable magnitudes, which makes the isotropic prior a sensible assumption. But the benefit is broader than MAP estimation: comparable scales also improve the *conditioning* of the optimization (recall the Hessian $\mathbf{H} = \frac{1}{N} \mathbf{X}^\top \mathbf{S} \mathbf{X}$), so first-order methods like SGD converge faster even without regularization.

1.4 Robust Logistic Regression

Sometimes we have **outliers** in our data, which are often due to labeling errors, also called **label noise**. To prevent the mode from begin adversely affected by such contamination, we will use **robust logistic regression**.

1.4.1 Mixture Model for the likelihood

Robust logistic regression. Outliers (misabeled or anomalous points) can distort a logistic regression fit. A simple fix modifies the *likelihood* so that each label y is generated uniformly at random with probability π , and otherwise by the usual conditional model. In the binary case,

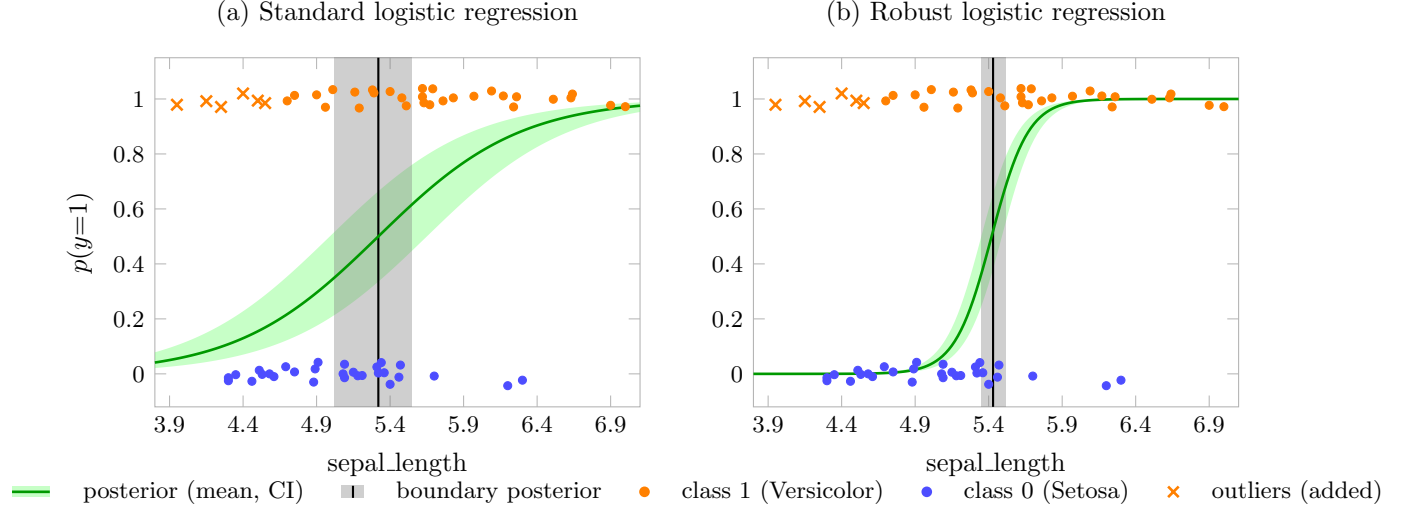
$$p(y \mid \mathbf{x}) = \pi \text{Ber}(y \mid 0.5) + (1 - \pi) \text{Ber}(y \mid \sigma(\mathbf{w}^\top \mathbf{x})).$$

The first component is a “junk” uniform model that absorbs outliers; the second is the standard logistic model. This **mixture observation model** trick is general — it can make many models robust, including DNNs.

Fitting. The model is fit with standard methods: SGD for point estimation, or Bayesian inference such as MCMC for the posterior.

Illustration (contaminated Iris). Take the 1d two-class Iris data and *contaminate* it by adding 6 class-1 (Versicolor) examples with abnormally low sepal length.

- **Standard model** (Fig. 10.10a): the outliers pull the decision boundary off and inflate posterior uncertainty about its location.
- **Robust model** (Fig. 10.10b): the boundary stays close to the one inferred from *clean* data (Fig. 4.20b), and the posterior uncertainty about its location is *smaller*.



1.4.2 Bi-tempered Loss

The mixture-likelihood approach handles outliers crudely. A more refined method (Amid et al. 2019) addresses the *two distinct ways* a mislabeled point hurts a convex-loss model, fixing each with a separate temperature parameter.

Problem 1: outliers far from the boundary. With a convex loss, a mislabeled point far from the boundary incurs an unbounded penalty and so distorts the fit. Fix: replace the log in cross entropy with a **tempered log**, using $0 \leq t_1 < 1$,

$$\log_t(x) \triangleq \frac{1}{1-t}(x^{1-t} - 1),$$

which is monotonically increasing and concave, and recovers the natural log as $t \rightarrow 1$. Crucially, for $0 \leq t < 1$ it is bounded below by $-1/(1-t)$, so the resulting loss is *bounded above* — a single outlier can do only limited damage. The **tempered cross entropy** is

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = \sum_c \left[y_c (\log_{t_1} y_c - \log_{t_1} \hat{y}_c) - \frac{1}{2-t_1} (y_c^{2-t_1} - \hat{y}_c^{2-t_1}) \right],$$

which, for a one-hot $\mathbf{y}$ with all mass on class c , simplifies to

$$\mathcal{L}(c, \hat{\mathbf{y}}) = -\log_{t_1} \hat{y}_c - \frac{1}{2-t_1} \left(1 - \sum_{c'=1}^C \hat{y}_{c'}^{2-t_1} \right).$$

At $t_1 = 1$ this reduces to standard cross entropy.

Problem 2: outliers near the boundary. The softmax is built on the exponential, so its tails decay too fast: a mislabeled point near the boundary dominates its local neighborhood. Fix: use a **tempered softmax** with heavier tails, parameterized by $t_2 > 1 > t_1$,

$$\hat{y}_c = \exp_{t_2}(a_c - \lambda_{t_2}(\mathbf{a})), \quad \exp_t(x) \triangleq [1 + (1-t)x]_+^{1/(1-t)},$$

where $\mathbf{a}$ is the logit vector and $[\cdot]_+ = \max(0, \cdot)$. The tempered exponential recovers the standard exponential as $t \rightarrow 1$, so the tempered softmax recovers the ordinary softmax

$$\hat{y}_c = \frac{\exp(a_c)}{\sum_{c'=1}^C \exp(a_{c'})}$$

at $t_2 = 1$.

Normalization constant. The shift $\lambda_{t_2}(\mathbf{a})$ enforces $\sum_c \hat{y}_c = 1$, i.e. it solves the fixed-point equation

$$\sum_{c=1}^C \exp_{t_2}(a_c - \lambda(\mathbf{a})) = 1.$$

Unlike the softmax's $\log \sum \exp$, this has no closed form; solve it by binary search or the iteration in Algorithm 10.2.

Algorithm 1 Iterative computation of $\lambda(\mathbf{a})$ (Amid et al. 2019)

```

1: Input: logits  $\mathbf{a}$ , temperature  $t > 1$ 
2:  $\mu := \max(\mathbf{a})$ 
3:  $\mathbf{a} := \mathbf{a} - \mu$ 
4: while  $\mathbf{a}$  not converged do
5:    $Z(\mathbf{a}) := \sum_{c=1}^C \exp_t(a_c)$ 
6:    $\mathbf{a} := Z(\mathbf{a})^{1-t}(\mathbf{a} - \mu \mathbf{1})$ 
7: end while
8: Return  $-\log_t \frac{1}{Z(\mathbf{a})} + \mu$ 

```

Bi-tempered logistic regression. Combining the tempered cross entropy (t_1) with the tempered softmax (t_2) gives **bi-tempered logistic regression**. The two temperatures target the two failure modes independently:

- Label noise *near* the boundary $\Rightarrow$ heavier tails: $t_1 = 1, t_2 = 4$.
- Label noise *far* from the boundary $\Rightarrow$ bounded loss: $t_1 = 0.2, t_2 = 1$.
- *Both* kinds of noise $\Rightarrow t_1 = 0.2, t_2 = 4$.

Each reduces to standard logistic regression at $t_1 = t_2 = 1$.

1.5 Bayesian Logistic Regression

So far we have used *point estimates* of the parameters (the MLE or MAP). Sometimes we instead want the full posterior $p(\mathbf{w} \mid \mathcal{D})$, to capture our **uncertainty** about $\mathbf{w}$. This matters most when data is scarce and when acting on a wrong decision is costly — exactly the regime where a single point estimate can be overconfident.

Unlike linear regression, logistic regression has *no* conjugate prior, so the posterior cannot be computed in closed form. We therefore approximate it. Many approximate-inference algorithms exist; here we use one of the simplest, the **Laplace approximation** (more advanced methods are covered in the sequel, Murphy 2023).

1.5.1 Laplace Approximation

The Laplace approximation fits a Gaussian to the posterior by Taylor-expanding the log-posterior to second order around its mode. Let $\mathcal{E}(\mathbf{w}) = -\log p(\mathbf{w}, \mathcal{D})$ be the energy (unnormalized negative log-posterior). A second-order expansion about the MAP estimate $\hat{\mathbf{w}}$ gives

$$\mathcal{E}(\mathbf{w}) \approx \mathcal{E}(\hat{\mathbf{w}}) + \frac{1}{2}(\mathbf{w} - \hat{\mathbf{w}})^\top \mathbf{H}(\mathbf{w} - \hat{\mathbf{w}}),$$

where the first-order term vanishes ($\hat{\mathbf{w}}$ is a stationary point) and $\mathbf{H} = \nabla^2 \mathcal{E}(\mathbf{w})|_{\hat{\mathbf{w}}}$ is the Hessian at the mode. Exponentiating yields a Gaussian posterior approximation

$$p(\mathbf{w} \mid \mathcal{D}) \approx \mathcal{N}(\mathbf{w} \mid \hat{\mathbf{w}}, \mathbf{H}^{-1}).$$

So we get the posterior almost for free: $\hat{\mathbf{w}}$ is the usual MAP estimate, and $\mathbf{H}^{-1}$ (the inverse Hessian we already computed for IRLS/Newton) becomes the posterior covariance.

1.5.2 Posterior Predictive Distribution

The posterior over $\mathbf{w}$ is mainly useful for computing the **posterior predictive distribution**, which marginalizes out parameter uncertainty:

$$p(y = 1 \mid \mathbf{x}, \mathcal{D}) = \int \sigma(\mathbf{w}^\top \mathbf{x}) p(\mathbf{w} \mid \mathcal{D}) d\mathbf{w}.$$

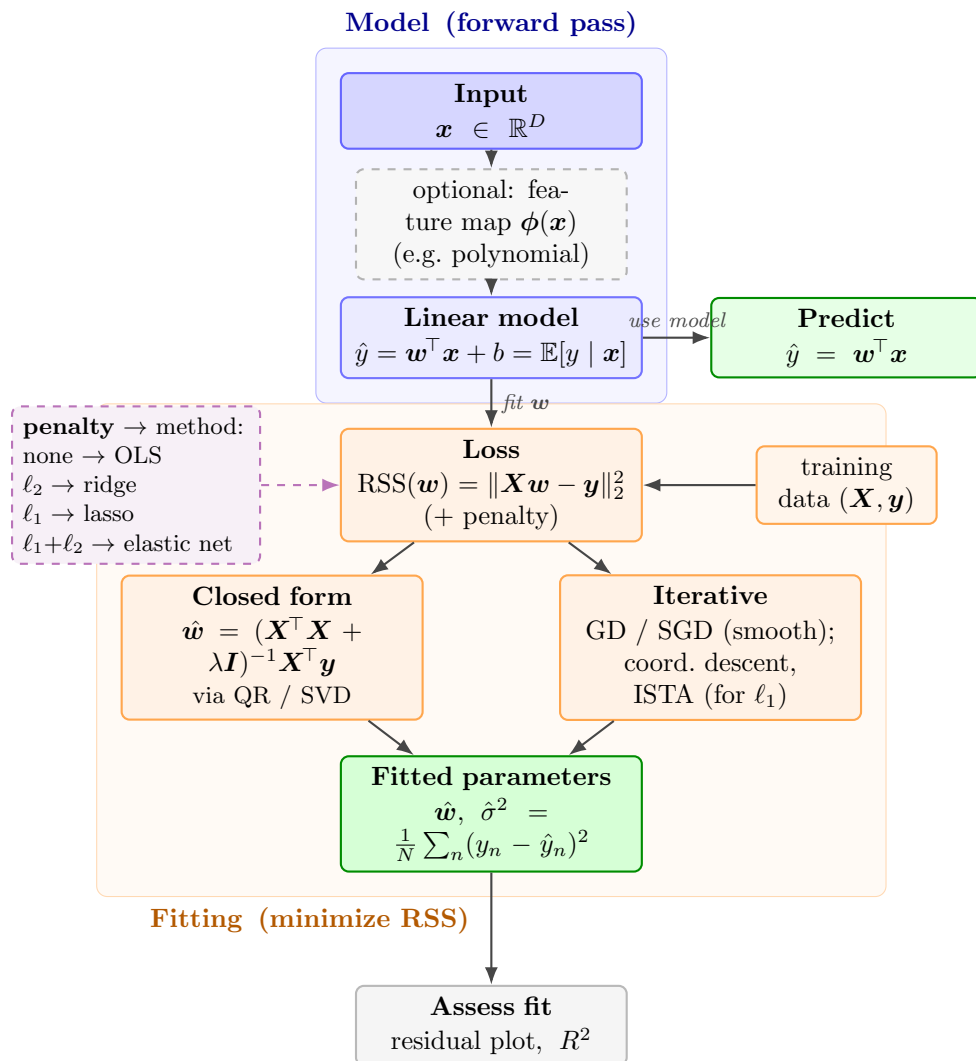
The simple **plug-in approximation** ignores this integral and uses the point estimate, $p(y = 1 \mid \mathbf{x}, \mathcal{D}) \approx \sigma(\hat{\mathbf{w}}^\top \mathbf{x})$, but this is *overconfident*: it produces probabilities near 0 or 1 even in regions where the data does not pin down the boundary. Integrating over the posterior instead *moderates* the predictions toward 0.5 where uncertainty is high.

Monte Carlo approximation. The integral has no closed form, but is easily approximated by sampling $\mathbf{w}_s \sim p(\mathbf{w} \mid \mathcal{D})$ and averaging:

$$p(y = 1 \mid \mathbf{x}, \mathcal{D}) \approx \frac{1}{S} \sum_{s=1}^S \sigma(\mathbf{w}_s^\top \mathbf{x}).$$

This is exactly the mechanism behind the credible bands in the robust-regression figure: each sampled $\mathbf{w}_s$ gives one sigmoid, and their spread is the posterior uncertainty about the decision boundary's location.

2 Linear Regression



Linear regression predicts a real-valued output $y \in \mathbb{R}$ from a vector of real-valued inputs $\mathbf{x} \in \mathbb{R}^D$.

Terminology.

- y : output, a.k.a. *dependent variable* / *target*.
- $\mathbf{x}$: input, a.k.a. *independent variables* / *explanatory variables* / *covariates*.
- $\mathbf{w}$: weights (regression coefficients).

Key assumption (defining property). The expected output is a *linear* function of the input:

$$\mathbb{E}[y | \mathbf{x}] = \mathbf{w}^\top \mathbf{x}.$$

- **Interpretable:** each weight w_d measures the effect of feature x_d on the output.
- **Easy to fit:** the linear form gives a simple (often closed-form) estimation problem.

Notes

- An intercept/bias b is usually included, $\mathbb{E}[y \mid \mathbf{x}] = \mathbf{w}^\top \mathbf{x} + b$, often absorbed by appending a constant 1 to $\mathbf{x}$.
- *Linear* refers to linearity in the **parameters** $\mathbf{w}$, not necessarily in the raw inputs — nonlinear extensions (e.g. basis-function expansions) come later.

2.1 Least Square Linear Regression

The most common form of linear regression model.

2.1.1 Terminology

The term **linear regression** usually refers to the model

$$p(y \mid \mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y \mid w_0 + \mathbf{w}^\top \mathbf{x}, \sigma^2),$$

with parameters $\boldsymbol{\theta} = (w_0, \mathbf{w}, \sigma^2)$. (In statistics, w_0 and $\mathbf{w}$ are usually written β_0 and $\boldsymbol{\beta}$.)

Weights / regression coefficients. The vector $\mathbf{w}_{1:D}$ holds the weights. Each w_d is the change in the output expected from increasing feature x_d by one unit. *Example:* if $x_1 = \text{age}$, $x_2 = \text{education level}$, $y = \text{income}$, then w_1 is the income gain per extra year of age and w_2 the gain per education level.

Bias / offset. The term w_0 is the **bias** (or offset): the output when all inputs are 0. It captures the unconditional mean $w_0 = \mathbb{E}[y]$ and acts as a baseline. We usually write $\mathbf{x} = [1, x_1, \dots, x_D]$ so that w_0 is absorbed into $\mathbf{w}$, giving the compact form $\mathbb{E}[y \mid \mathbf{x}] = \mathbf{w}^\top \mathbf{x}$.

Naming by dimension.

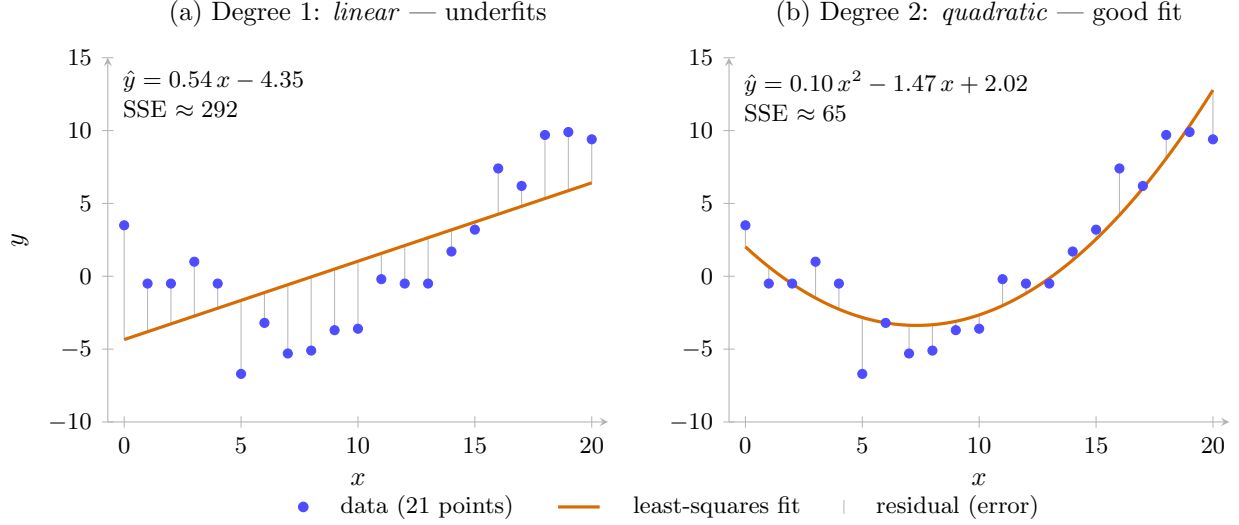
- $D = 1$: **simple linear regression**, $f(x; \mathbf{w}) = ax + b$, with intercept $b = w_0$ and slope $a = w_1$.
- $D > 1$: **multiple linear regression**.
- Output also multi-dimensional, $\mathbf{y} \in \mathbb{R}^J$ ($J > 1$): **multivariate linear regression**,

$$p(\mathbf{y} \mid \mathbf{x}, \mathbf{W}) = \prod_{j=1}^J \mathcal{N}(y_j \mid \mathbf{w}_j^\top \mathbf{x}, \sigma_j^2).$$

Nonlinear in the inputs, linear in the parameters. A straight line rarely fits well, but we can apply a fixed nonlinear feature map $\phi(\mathbf{x})$:

$$p(y \mid \mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y \mid \mathbf{w}^\top \phi(\mathbf{x}), \sigma^2).$$

As long as ϕ is fixed, the model stays *linear in the parameters* $\mathbf{w}$ even though it is nonlinear in $\mathbf{x}$. *Example (polynomial regression):* for 1d input and degree d , take $\phi(x) = [1, x, x^2, \dots, x^d]$ as show below.



2.2 Least Squares Linear Regression

2.2.1 Objective Function

Estimate the parameters by maximum likelihood, i.e. by minimizing the negative log-likelihood. Under the Gaussian model $p(y \mid \mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y \mid \mathbf{w}^\top \mathbf{x}, \sigma^2)$,

$$\text{NLL}(\mathbf{w}, \sigma^2) = - \sum_{n=1}^N \log \left[\left(\frac{1}{2\pi\sigma^2} \right)^{1/2} \exp \left(-\frac{1}{2\sigma^2} (y_n - \mathbf{w}^\top \mathbf{x}_n)^2 \right) \right] \quad (19)$$

$$= \frac{1}{2\sigma^2} \sum_{n=1}^N (y_n - \hat{y}_n)^2 + \frac{N}{2} \log(2\pi\sigma^2), \quad (20)$$

where $\hat{y}_n \triangleq \mathbf{w}^\top \mathbf{x}_n$ is the predicted response. The MLE is the stationary point $\nabla_{\mathbf{w}, \sigma} \text{NLL}(\mathbf{w}, \sigma^2) = \mathbf{0}$; we can optimize over $\mathbf{w}$ first, then solve for the optimal σ .

$$\text{NLL}(\mathbf{w}, \sigma^2) = \underbrace{\frac{1}{2\sigma^2}}_{\substack{\text{positive} \\ \text{scale factor} \\ \text{(const. in } \mathbf{w})}} \underbrace{\sum_{n=1}^N (y_n - \hat{y}_n)^2}_{\substack{\text{depends on } \mathbf{w} \\ = 2 \text{RSS}(\mathbf{w})}} + \underbrace{\frac{N}{2} \log(2\pi\sigma^2)}_{\substack{\text{additive constant} \\ \text{(const. in } \mathbf{w})}}.$$

Reduction to least squares. Focusing on the weights, the term $\frac{N}{2} \log(2\pi\sigma^2)$ is constant in $\mathbf{w}$, and $\frac{1}{2\sigma^2}$ is a positive scale factor. So minimizing the NLL over $\mathbf{w}$ is equivalent to minimizing the **residual sum of squares**:

$$\text{RSS}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N (y_n - \mathbf{w}^\top \mathbf{x}_n)^2 = \frac{1}{2} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 = \frac{1}{2} (\mathbf{X}\mathbf{w} - \mathbf{y})^\top (\mathbf{X}\mathbf{w} - \mathbf{y}).$$

This is why Gaussian-noise MLE and **least squares** give the same weight estimate $\hat{\mathbf{w}}$.

2.2.2 Ordinary Least Squares

The gradient is given by:

$$\nabla_{\mathbf{w}} \text{RSS}(\mathbf{w}) = \mathbf{X}^\top \mathbf{X} \mathbf{w} - \mathbf{X}^\top \mathbf{y}$$

Setting the gradient to 0 and solving gives.

$$\mathbf{X}^\top \mathbf{X} \mathbf{w} = \mathbf{X}^\top \mathbf{y}$$

This is the **normal equation**, since at the optimal solution, $\mathbf{y} - \mathbf{X}\mathbf{w}$ is normal (orthogonal) to the range of $\mathbf{X}$. Corresponding solution $\hat{\mathbf{w}}$ is the **ordinary least squares (OLS)** give by:

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Pseudo-inverse. The quantity

$$\mathbf{X}^\dagger = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$$

is the (left) **pseudo-inverse** of the non-square matrix $\mathbf{X}$, so the least-squares solution can be written compactly as $\hat{\mathbf{w}} = \mathbf{X}^\dagger \mathbf{y}$.

Uniqueness via the Hessian. The solution is unique if the objective is strictly convex, i.e. if its Hessian is positive definite. Here

$$\mathbf{H}(\mathbf{w}) = \frac{\partial^2}{\partial \mathbf{w}^2} \text{RSS}(\mathbf{w}) = \mathbf{X}^\top \mathbf{X}.$$

If $\mathbf{X}$ is *full rank* (its columns are linearly independent), then $\mathbf{H}$ is positive definite: for any $\mathbf{v} \neq \mathbf{0}$,

$$\mathbf{v}^\top (\mathbf{X}^\top \mathbf{X}) \mathbf{v} = (\mathbf{X} \mathbf{v})^\top (\mathbf{X} \mathbf{v}) = \|\mathbf{X} \mathbf{v}\|_2^2 > 0,$$

where the strict inequality holds because full rank forces $\mathbf{X} \mathbf{v} \neq \mathbf{0}$ for $\mathbf{v} \neq \mathbf{0}$. Hence in the full-rank case the least-squares objective has a unique global minimum.

2.2.3 Geometric Interpretation

The normal equations have an elegant geometric reading as an **orthogonal projection**. Assume $N > D$ (more observations than unknowns), so the system is **overdetermined** — generally no $\mathbf{w}$ makes $\mathbf{X}\mathbf{w} = \mathbf{y}$ exactly.

Setup. We seek the vector $\hat{\mathbf{y}} \in \mathbb{R}^N$ that lies in the subspace spanned by the columns of $\mathbf{X}$ and is closest to $\mathbf{y}$:

$$\underset{\hat{\mathbf{y}} \in \text{span}(\{\mathbf{x}_{:,1}, \dots, \mathbf{x}_{:,D}\})}{\text{argmin}} \quad \|\mathbf{y} - \hat{\mathbf{y}}\|_2,$$

where $\mathbf{x}_{:,d}$ is the d th column of $\mathbf{X}$. Since $\hat{\mathbf{y}} \in \text{span}(\mathbf{X})$, there is a weight vector $\mathbf{w}$ with

$$\hat{\mathbf{y}} = w_1 \mathbf{x}_{:,1} + \dots + w_D \mathbf{x}_{:,D} = \mathbf{X} \mathbf{w}.$$

Orthogonality condition. The residual $\mathbf{y} - \hat{\mathbf{y}}$ has minimum norm exactly when it is *orthogonal to every column* of $\mathbf{X}$. This recovers the normal equations:

$$\mathbf{x}_{:,d}^\top (\mathbf{y} - \hat{\mathbf{y}}) = 0 \implies \mathbf{X}^\top (\mathbf{y} - \mathbf{X} \mathbf{w}) = \mathbf{0} \implies \hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

Projection / hat matrix. The fitted values are therefore

$$\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{w}} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y},$$

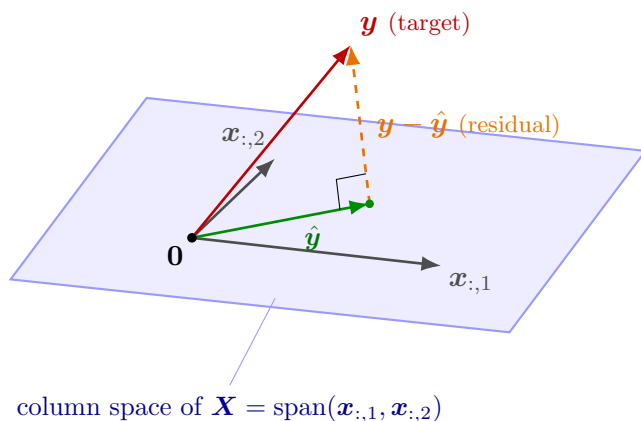
i.e. the *orthogonal projection* of $\mathbf{y}$ onto the column space of $\mathbf{X}$. The matrix

$$\text{Proj}(\mathbf{X}) \triangleq \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top, \quad \hat{\mathbf{y}} = \text{Proj}(\mathbf{X}) \mathbf{y},$$

is the **hat matrix** (it “puts the hat on” $\mathbf{y}$). With $N = 3$, $D = 2$, the two columns of $\mathbf{X}$ span a 2d plane in $\mathbb{R}^3$, and $\hat{\mathbf{y}}$ is the foot of the perpendicular dropped from $\mathbf{y}$ onto that plane.

Special case (projection onto a line). If $\mathbf{X} = \mathbf{x}$ is a single column vector, projecting $\mathbf{y}$ onto the line through $\mathbf{x}$ gives

$$\text{Proj}(\mathbf{x}) \mathbf{y} = \mathbf{x} \frac{\mathbf{x}^\top \mathbf{y}}{\mathbf{x}^\top \mathbf{x}}.$$



2.2.4 Solving the Normal Equations Numerically

Recall the OLS solution

$$\hat{\mathbf{w}} = \mathbf{X}^\dagger \mathbf{y} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

Although this can in principle be computed by forming and inverting $\mathbf{X}^\top \mathbf{X}$, we *should not* do so: $\mathbf{X}^\top \mathbf{X}$ is often **ill-conditioned or singular**, so explicit inversion is numerically unstable. (Squaring $\mathbf{X}$ squares its condition number, amplifying any sensitivity.) Better approaches solve the least-squares system $\mathbf{X}\mathbf{w} = \mathbf{y}$ directly via matrix factorizations.

SVD (default, most general). Compute the pseudo-inverse from the singular value decomposition. This is the most robust method and is what production code uses — e.g. `sklearn`’s linear regression calls `scipy.linalg.lstsq`, which calls LAPACK’s SVD-based DGELSD routine.

QR (faster when $N \gg D$). For tall-skinny $\mathbf{X}$, QR decomposition is quicker. Writing $\mathbf{X} = \mathbf{Q}\mathbf{R}$ with $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$ and $\mathbf{R}$ upper triangular,

$$(\mathbf{Q}\mathbf{R})\mathbf{w} = \mathbf{y} \tag{21}$$

$$\mathbf{Q}^\top \mathbf{Q}\mathbf{R}\mathbf{w} = \mathbf{Q}^\top \mathbf{y} \tag{22}$$

$$\mathbf{w} = \mathbf{R}^{-1}(\mathbf{Q}^\top \mathbf{y}). \tag{23}$$

Because $\mathbf{R}$ is upper triangular, the last step is solved by **back-substitution** — no matrix inversion required. (If $N = D$ and $\mathbf{X}$ is full rank, the system has a unique exact solution with zero error.)

Iterative solvers (large / sparse $\mathbf{X}$). As an alternative to direct factorizations:

- **Conjugate gradient (CG):** requires $\mathbf{X}$ symmetric positive definite.
- **GMRES** (generalized minimal residual): works for general $\mathbf{X}$ (`scipy.sparse.linalg.gmres`).

These need only matrix–vector products (a *linear operator*), making them well-suited to sparse or structured $\mathbf{X}$.

Standardize first. Regardless of solver, it is essential to **standardize** the inputs (zero mean, unit variance) before fitting, using the formula.

2.3 Weighted Least Squares

So far we assumed constant noise variance σ^2 (homoscedasticity). If instead the noise depends on the input (heteroscedasticity), the model becomes

$$p(y \mid \mathbf{x}; \boldsymbol{\theta}) = \mathcal{N}(y \mid \mathbf{w}^\top \mathbf{x}, \sigma^2(\mathbf{x})) = \frac{1}{\sqrt{2\pi\sigma^2(\mathbf{x})}} \exp\left(-\frac{1}{2\sigma^2(\mathbf{x})}(y - \mathbf{w}^\top \mathbf{x})^2\right).$$

Stacking over the dataset, this is a Gaussian with a diagonal (non-isotropic) covariance,

$$p(\mathbf{y} \mid \mathbf{X}; \boldsymbol{\theta}) = \mathcal{N}(\mathbf{y} \mid \mathbf{X}\mathbf{w}, \boldsymbol{\Lambda}^{-1}), \quad \boldsymbol{\Lambda} = \text{diag}(1/\sigma^2(\mathbf{x}_n)),$$

which is known as **weighted linear regression**. The matrix $\boldsymbol{\Lambda}$ is the *precision* (inverse variance), so each example is weighted by how reliable it is: low-noise points get large weight, high-noise points get small weight.

Weighted least squares estimate. Maximizing the likelihood gives

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \boldsymbol{\Lambda} \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{\Lambda} \mathbf{y},$$

the **weighted least squares (WLS)** estimate. Setting $\boldsymbol{\Lambda} = \mathbf{I}$ recovers ordinary least squares

2.3.1 Other Approaches to Computing the MLE

Besides solving the normal equations directly, several alternative routes to the OLS estimate are useful in special cases.

Offset and slope separately. For the model $\mathcal{N}(y \mid w_0 + \mathbf{w}^\top \mathbf{x}, \sigma^2)$, we can fit $(w_0, \mathbf{w})$ jointly by appending a column of 1s to $\mathbf{X}$ — or solve for them *separately* by centering the data:

$$\hat{\mathbf{w}} = (\mathbf{X}_c^\top \mathbf{X}_c)^{-1} \mathbf{X}_c^\top \mathbf{y}_c = \left[\sum_n (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^\top \right]^{-1} \left[\sum_n (y_n - \bar{y})(\mathbf{x}_n - \bar{\mathbf{x}}) \right], \quad \hat{w}_0 = \bar{y} - \bar{\mathbf{x}}^\top \hat{\mathbf{w}},$$

where $\mathbf{X}_c$ has the centered rows $\mathbf{x}_n - \bar{\mathbf{x}}$ and $\mathbf{y}_c = \mathbf{y} - \bar{y}$. So: fit $\hat{\mathbf{w}}$ on centered data, then read off the intercept as $\hat{w}_0 = \bar{y} - \bar{\mathbf{x}}^\top \hat{\mathbf{w}}$. (Equivalently, in the form $\hat{y} = \beta_0 + \boldsymbol{\beta}^\top (\mathbf{x} - \bar{\mathbf{x}})$ the intercept is just $\hat{\beta}_0 = \bar{y}$.)

Simple linear regression (1d). For scalar x this reduces to the familiar formulas:

$$\hat{w}_1 = \frac{\sum_n (x_n - \bar{x})(y_n - \bar{y})}{\sum_n (x_n - \bar{x})^2} = \frac{C_{xy}}{C_{xx}} = \rho_{xy} \frac{\sigma_y}{\sigma_x}, \quad \hat{w}_0 = \bar{y} - \hat{w}_1 \bar{x},$$

with $C_{xx} = \mathbb{V}[X] = \sigma_x^2$, $C_{xy} = \text{Cov}[X, Y]$, and $\rho_{xy} = C_{xy}/(\sigma_x \sigma_y)$. The prediction has a clean standardized form,

$$\frac{\hat{y} - \bar{y}}{\sigma_y} = \rho_{xy} \frac{x - \bar{x}}{\sigma_x},$$

read as: measure how many standard deviations x sits from $\bar{x}$, then predict $\bar{y}$ plus ρ_{xy} times that many standard deviations of Y .

Partial regression. The regression coefficient of Y on X is the slope

$$R_{YX} \triangleq \frac{\partial}{\partial x} \mathbb{E}[Y \mid X = x] = w_1 = \frac{C_{xy}}{C_{xx}}.$$

With two inputs, $Y = w_0 + w_1 X_1 + w_2 X_2 + \epsilon$ ($\mathbb{E}[\epsilon] = 0$), the optimal w_1 is the **partial regression coefficient** of Y on X_1 holding X_2 fixed,

$$w_1 = R_{YX_1 \cdot X_2} = \frac{\partial}{\partial x} \mathbb{E}[Y \mid X_1 = x, X_2],$$

which is invariant to the value of X_2 conditioned on. This generalizes to any number of inputs and gives the key interpretation: $\hat{w}_j$ is the change in y per unit change in x_j *with all other inputs held constant*.

Recursive (online) MLE. OLS is a *batch* method; for streaming data we update the 1d estimate recursively via sufficient statistics. With running means $\bar{x}^{(n)}, \bar{y}^{(n)}$ and second moments $C_{xx}^{(n)}, C_{xy}^{(n)}$, the means update as

$$\bar{x}^{(n+1)} = \bar{x}^{(n)} + \frac{1}{n+1} (x_{n+1} - \bar{x}^{(n)}), \quad \bar{y}^{(n+1)} = \bar{y}^{(n)} + \frac{1}{n+1} (y_{n+1} - \bar{y}^{(n)}).$$

Using the identity $C_{xy}^{(n)} = \frac{1}{n} (\sum_i x_i y_i - n \bar{x}^{(n)} \bar{y}^{(n)})$, the covariance updates as

$$C_{xy}^{(n+1)} = \frac{1}{n+1} \left[x_{n+1} y_{n+1} + n C_{xy}^{(n)} + n \bar{x}^{(n)} \bar{y}^{(n)} - (n+1) \bar{x}^{(n+1)} \bar{y}^{(n+1)} \right],$$

and $C_{xx}^{(n+1)}$ analogously. Then $\hat{w}_1 = C_{xy}/C_{xx}$, $\hat{w}_0 = \bar{y} - \hat{w}_1 \bar{x}$ as before (Fig. 11.4). For D -dimensional inputs the simplest online approach is SGD, giving the **least mean squares (LMS)** algorithm.

Generative derivation. Linear regression is *discriminative* ($p(y \mid \mathbf{x})$), but we can also fit the *joint* $p(\mathbf{x}, y)$ and condition. Modelling $p(\mathbf{x}, y)$ as an MVN, the MLEs are the empirical moments $\boldsymbol{\mu}_x, \mu_y, \boldsymbol{\Sigma}_{xx} = \frac{1}{N} \mathbf{X}_c^\top \mathbf{X}_c, \boldsymbol{\Sigma}_{xy} = \frac{1}{N} \mathbf{X}_c^\top \mathbf{y}_c$. The Gaussian conditioning formula gives

$$\mathbb{E}[y \mid \mathbf{x}] = \mu_y + \boldsymbol{\Sigma}_{xy}^\top \boldsymbol{\Sigma}_{xx}^{-1} (\mathbf{x} - \boldsymbol{\mu}_x) = w_0 + \mathbf{w}^\top \mathbf{x}, \quad \text{with} \quad \mathbf{w} = \boldsymbol{\Sigma}_{xx}^{-1} \boldsymbol{\Sigma}_{xy} = (\mathbf{X}_c^\top \mathbf{X}_c)^{-1} \mathbf{X}_c^\top \mathbf{y}_c, \quad w_0 = \bar{y} - \mathbf{w}^\top \bar{\mathbf{x}}.$$

This is identical to the discriminative MLE above: fitting the joint and conditioning equals fitting the conditional directly — but *only* for Gaussian models.

MLE for the noise variance. Having estimated $\hat{\mathbf{w}}$, the variance MLE is

$$\hat{\sigma}_{\text{mle}}^2 = \underset{\sigma^2}{\text{argmin}} \text{NLL}(\hat{\mathbf{w}}, \sigma^2) = \frac{1}{N} \sum_{n=1}^N (y_n - \mathbf{x}_n^\top \hat{\mathbf{w}})^2,$$

simply the mean-squared error of the residuals.

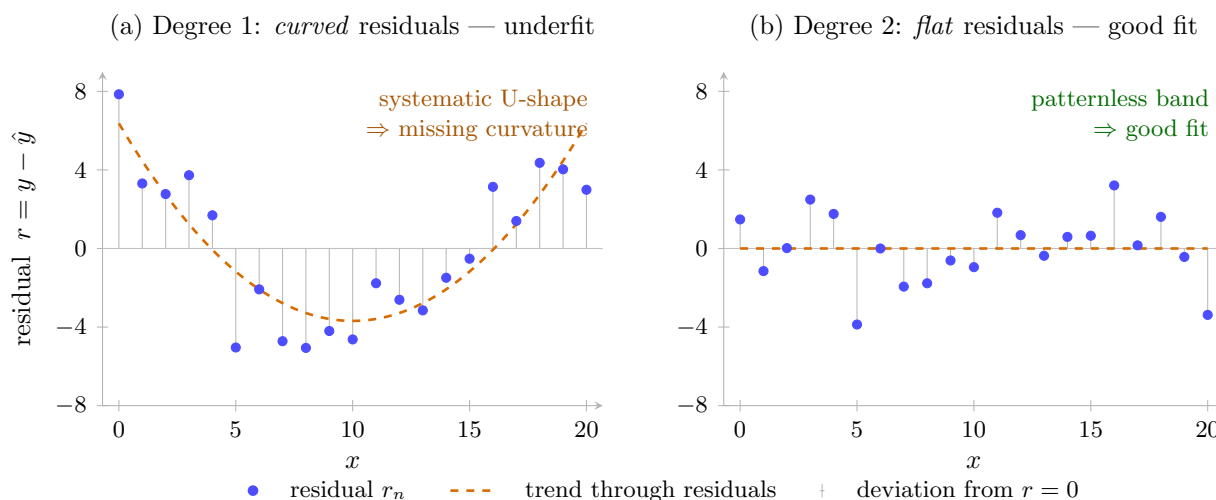
2.4 Measuring Goodness of Fit

Simple ways to assess how well a regression model fits the data (**goodness of fit**).

2.4.1 Residual Plots

Residual plots (1d inputs). A simple check on the model is to plot the residuals $r_n = y_n - \hat{y}_n$ against the input x_n . The Gaussian-noise assumption says the residuals are $\mathcal{N}(0, \sigma^2)$, so a well-fit model produces a structureless cloud: points scattered roughly evenly above and below the line $r = 0$, with no visible trend. Any systematic pattern signals a *lack of fit*. For the linear fit (Fig. 11.5a) the residuals show a curved trend — the model is missing the curvature — whereas the quadratic fit (Fig. 11.5b) leaves a flat, patternless band.

Fit-vs-actual plots (general inputs). Residual-vs- x_n plots only work for 1d inputs. For multi-dimensional $\mathbf{x}$, instead plot the predictions $\hat{y}_n$ against the true outputs y_n . A good model has points lying close to the diagonal $\hat{y} = y$; scatter away from the diagonal indicates error (Fig. 11.6).



2.4.2 Prediction Accuracy and R^2

We can score the fit quantitatively. The basic quantity is the **residual sum of squares**

$$\text{RSS}(\mathbf{w}) = \sum_{n=1}^N (y_n - \mathbf{w}^\top \mathbf{x}_n)^2,$$

where lower is better. Its scale depends on N , so a normalized version is the **root mean squared error**

$$\text{RMSE}(\mathbf{w}) \triangleq \sqrt{\frac{1}{N} \text{RSS}(\mathbf{w})},$$

which is in the same units as y .

Coefficient of determination (R^2). A more interpretable, scale-free measure:

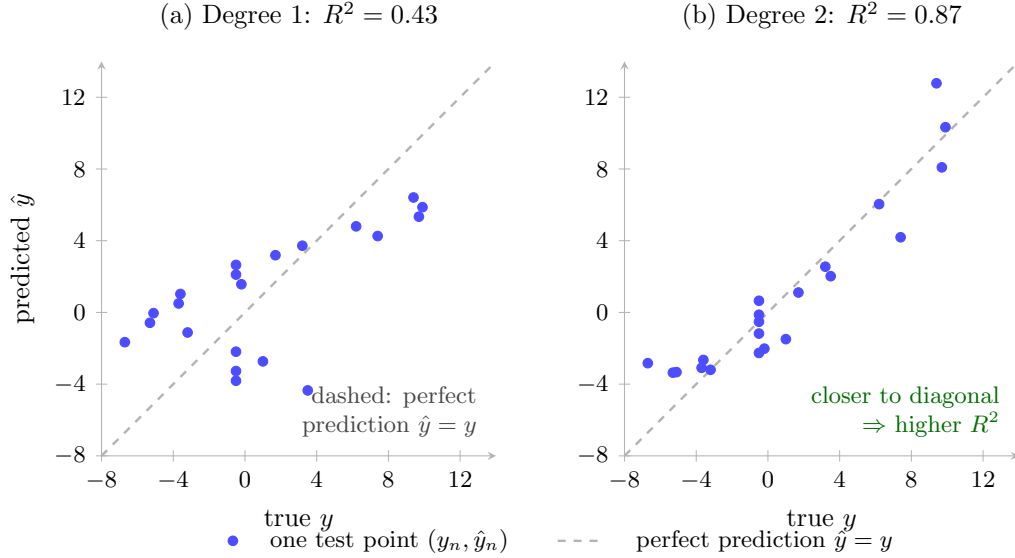
$$R^2 \triangleq 1 - \frac{\sum_{n=1}^N (\hat{y}_n - y_n)^2}{\sum_{n=1}^N (\bar{y} - y_n)^2} = 1 - \frac{\text{RSS}}{\text{TSS}},$$

where $\bar{y} = \frac{1}{N} \sum_n y_n$, $\text{RSS} = \sum_n (y_n - \hat{y}_n)^2$ is the residual sum of squares, and $\text{TSS} = \sum_n (y_n - \bar{y})^2$ is the **total sum of squares**.

Interpretation. R^2 compares the model against the trivial baseline of always predicting the mean $\hat{y}_n = \bar{y}$ (whose error is exactly TSS):

- $R^2 = 0$: the model does *no better* than predicting the mean.
- $R^2 = 1$: the model fits perfectly ($\text{RSS} = 0$).
- larger R^2 : greater fraction of the output variance explained (better fit).

So R^2 is the fraction of variance in y explained by the model (Fig. 11.6).



2.5 Ridge Regression

Maximum likelihood overfits when the model is flexible relative to the data. A simple fix is **MAP estimation** with a zero-mean Gaussian prior on the weights, $p(\mathbf{w}) = \mathcal{N}(\mathbf{w} \mid \mathbf{0}, \lambda^{-1} \mathbf{I})$. This is called **ridge regression**.

MAP objective. The MAP estimate minimizes the (negative log) posterior, which is the data NLL plus the prior penalty:

$$\hat{\mathbf{w}}_{\text{map}} = \underset{\mathbf{w}}{\text{argmin}} \frac{1}{2\sigma^2} (\mathbf{y} - \mathbf{X}\mathbf{w})^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) + \frac{1}{2\tau^2} \mathbf{w}^\top \mathbf{w} \quad (24)$$

$$= \underset{\mathbf{w}}{\text{argmin}} \text{RSS}(\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2, \quad (25)$$

where the two variance terms collapse into a single **regularization strength**

$$\lambda \triangleq \frac{\sigma^2}{\tau^2},$$

proportional to how tight the prior is (τ^2 is the prior variance, σ^2 the noise variance), and

$$\|\mathbf{w}\|_2 \triangleq \sqrt{\sum_{d=1}^D |w_d|^2} = \sqrt{\mathbf{w}^\top \mathbf{w}}$$

is the ℓ_2 norm.

Interpretation. The penalty $\lambda\|\mathbf{w}\|_2^2$ discourages large-magnitude weights, shrinking them toward zero. This is ℓ_2 **regularization** (a.k.a. **weight decay**). Larger λ means a tighter prior and a less flexible model; $\lambda = 0$ recovers OLS.

Note on the bias. We do *not* penalize the offset w_0 : it only sets the global mean of the output and does not contribute to overfitting.

2.5.1 Computing the MAP Estimate

Closed form. The ridge MAP estimate minimizes the penalized objective

$$J(\mathbf{w}) = (\mathbf{y} - \mathbf{X}\mathbf{w})^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda\|\mathbf{w}\|_2^2, \quad \lambda = \sigma^2/\tau^2.$$

Setting the gradient to zero,

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = 2(\mathbf{X}^\top \mathbf{X}\mathbf{w} - \mathbf{X}^\top \mathbf{y} + \lambda\mathbf{w}) = \mathbf{0},$$

gives the **ridge solution**

$$\hat{\mathbf{w}}_{\text{map}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_D)^{-1} \mathbf{X}^\top \mathbf{y}.$$

Compared with OLS, the only change is the added $\lambda \mathbf{I}_D$ on the diagonal — the “ridge.” This also guarantees the matrix is invertible (it makes $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}$ positive definite even when $\mathbf{X}^\top \mathbf{X}$ is singular), curing the ill-conditioning flagged.

As before, explicitly forming and inverting $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}$ is slow and numerically unstable, so we use a factorization instead.

QR via virtual data ($N \geq D$). The penalty can be emulated by appending “virtual” data points to the training set, turning ridge into a *standard* least-squares problem. With prior $p(\mathbf{w}) = \mathcal{N}(\mathbf{0}, \Lambda^{-1})$ (for ridge, $\Lambda = \frac{1}{\tau^2} \mathbf{I}$), augment

$$\tilde{\mathbf{X}} = \begin{pmatrix} \mathbf{X}/\sigma \\ \sqrt{\Lambda} \end{pmatrix} \in \mathbb{R}^{(N+D) \times D}, \quad \tilde{\mathbf{y}} = \begin{pmatrix} \mathbf{y}/\sigma \\ \mathbf{0}_{D \times 1} \end{pmatrix},$$

where $\sqrt{\Lambda}$ is a Cholesky factor of Λ ($\Lambda = \sqrt{\Lambda} \sqrt{\Lambda}^\top$). The plain RSS on this expanded data equals the penalized RSS on the original data:

$$(\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\mathbf{w})^\top (\tilde{\mathbf{y}} - \tilde{\mathbf{X}}\mathbf{w}) = \frac{1}{\sigma^2} (\mathbf{y} - \mathbf{X}\mathbf{w})^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) + \mathbf{w}^\top \Lambda \mathbf{w}.$$

So $\hat{\mathbf{w}}_{\text{map}} = (\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^\top \tilde{\mathbf{y}}$ is an ordinary OLS fit, solved by QR on $\tilde{\mathbf{X}}$ in $O((N+D)D^2)$ time.

SVD ($D > N$). When there are more features than examples (the typical ridge regime), SVD is faster. Let $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$ and set $\mathbf{R} = \mathbf{U}\mathbf{S}$ (an $N \times N$ matrix). Then

$$\hat{\mathbf{w}}_{\text{map}} = \mathbf{V}(\mathbf{R}^\top \mathbf{R} + \lambda \mathbf{I}_N)^{-1} \mathbf{R}^\top \mathbf{y}.$$

This effectively replaces each D -dimensional input $\mathbf{x}_i$ with an N -dimensional vector $\mathbf{r}_i$ and does the penalized fit in that smaller space, costing $O(DN^2)$ — cheaper than the $O(D^3)$ of the naive primal solve when $D > N$.

2.5.2 Connection Between Ridge Regression and PCA

There is a clean SVD view of ridge that explains *why* it helps. Let $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$ be the SVD with singular values σ_j . Plugging into the ridge solution, the fitted training values are

$$\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{w}}_{\text{map}} = \sum_{j=1}^D \mathbf{u}_j \underbrace{\frac{\sigma_j^2}{\sigma_j^2 + \lambda}}_{\text{filter factor}} \mathbf{u}_j^\top \mathbf{y},$$

whereas ordinary least squares gives

$$\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{w}}_{\text{mle}} = \sum_{j=1}^D \mathbf{u}_j \mathbf{u}_j^\top \mathbf{y}.$$

So OLS keeps every direction $\mathbf{u}_j$ at full strength, while ridge multiplies each by a **filter factor** $\sigma_j^2/(\sigma_j^2 + \lambda) \in [0, 1]$:

- $\sigma_j^2 \gg \lambda$: factor ≈ 1 — direction kept (well-determined).
- $\sigma_j^2 \ll \lambda$: factor ≈ 0 — direction downweighted (ill-determined).

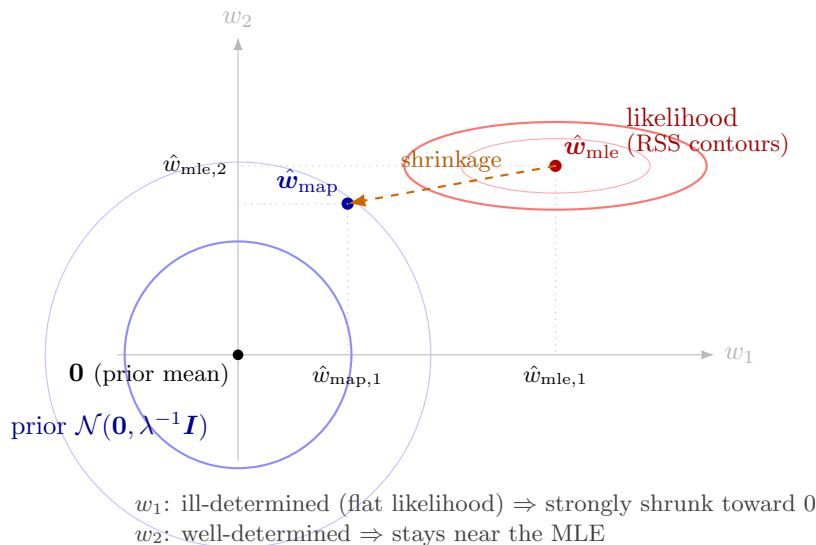
Effective degrees of freedom. Summing the filter factors gives the effective complexity of the model,

$$\text{dof}(\lambda) = \sum_{j=1}^D \frac{\sigma_j^2}{\sigma_j^2 + \lambda},$$

which equals D at $\lambda = 0$ (OLS) and decreases to 0 as $\lambda \rightarrow \infty$.

Why downweight small- σ_j directions (shrinkage). With a flat prior, $\text{Cov}[\mathbf{w} \mid \mathcal{D}] \propto (\mathbf{X}^\top \mathbf{X})^{-1}$, so the directions of *greatest posterior uncertainty* are the eigenvectors of $\mathbf{X}^\top \mathbf{X}$ with the *smallest* eigenvalues — i.e. the smallest singular values σ_j . Ridge shrinks exactly those poorly-determined directions toward the prior mean $\mathbf{0}$, while leaving well-determined directions near their MLE values. This selective pulling-toward-zero is called **shrinkage** (Fig. 11.7).

Contrast with principal components regression (PCR). PCR is a “hard” version: keep only the top K principal directions and discard the remaining $D - K$ entirely. Ridge instead applies a “soft” weighting $\sigma_j^2/(\sigma_j^2 + \lambda)$ to *all* directions. Because PCR throws information away abruptly while ridge tapers it, ridge usually predicts better.



2.5.3 Choosing the Strength of the Regularizer

The regularization strength λ is a hyperparameter, not fit by the training objective itself. Two main approaches:

Cross-validation. Try a finite grid of candidate λ values and use cross-validation to estimate the expected loss of each, picking the best. This is expensive when the grid is large.

Warm starting / regularization path. The cost is reduced by *warm starting*: use the solution $\hat{\mathbf{w}}(\lambda_k)$ as the initializer for the next fit $\hat{\mathbf{w}}(\lambda_{k+1})$ with $\lambda_{k+1} < \lambda_k$. That is, start from a heavily constrained model (large λ) and gradually relax the penalty. The sequence of solutions $\{\hat{\mathbf{w}}_k\}$ swept out this way is the **regularization path**.

Empirical Bayes. Alternatively, choose λ by maximizing the **marginal likelihood** (evidence):

$$\hat{\lambda} = \underset{\lambda}{\operatorname{argmax}} \log p(\mathcal{D} \mid \lambda).$$

This gives essentially the same answer as CV (Fig. 4.7b), but with two advantages: it needs only a *single* model fit (vs. K fits for K -fold CV), and $p(\mathcal{D} \mid \lambda)$ is a *smooth* function of λ , so gradient-based optimization can be used instead of discrete grid search.

2.6 Lasso Regression

Ridge regression uses a Gaussian prior, which keeps the weights *small* and prevents overfitting. Sometimes, though, we want the weights to be not just small but *exactly zero*, i.e. we want $\hat{\mathbf{w}}$ to be **sparse**. The natural measure of sparsity is the ℓ_0 “norm,” which counts the nonzero entries:

$$\|\mathbf{w}\|_0 = \sum_{d=1}^D \mathbb{I}(|w_d| > 0).$$

Why sparsity = feature selection. The prediction is $f(\mathbf{x}; \mathbf{w}) = \sum_{d=1}^D w_d x_d$, so if $w_d = 0$, feature x_d drops out of the model entirely. Driving weights to exactly zero therefore performs

feature selection: the surviving nonzero weights identify the relevant features. (The same trick applies to nonlinear models such as DNNs, by encouraging sparse first-layer weights.)

2.6.1 MAP Estimation with a Laplace Prior (ℓ_1 Regularization)

To get sparse estimates, we do MAP estimation with a **Laplace prior** on the weights instead of a Gaussian:

$$p(\mathbf{w} \mid \lambda) = \prod_{d=1}^D \text{Laplace}(w_d \mid 0, 1/\lambda) \propto \prod_{d=1}^D e^{-\lambda|w_d|}, \quad \text{Laplace}(w \mid \mu, b) \triangleq \frac{1}{2b} \exp\left(-\frac{|w - \mu|}{b}\right),$$

where λ is the sparsity parameter, μ the location, and $b > 0$ the scale. The key property: $\text{Laplace}(w \mid 0, b)$ puts *more* density at 0 (and has a sharp peak there) than $\mathcal{N}(w \mid 0, \sigma^2)$ with the same variance — which is what pushes weights to be exactly zero.

Lasso objective. Taking the negative log-posterior, MAP estimation minimizes

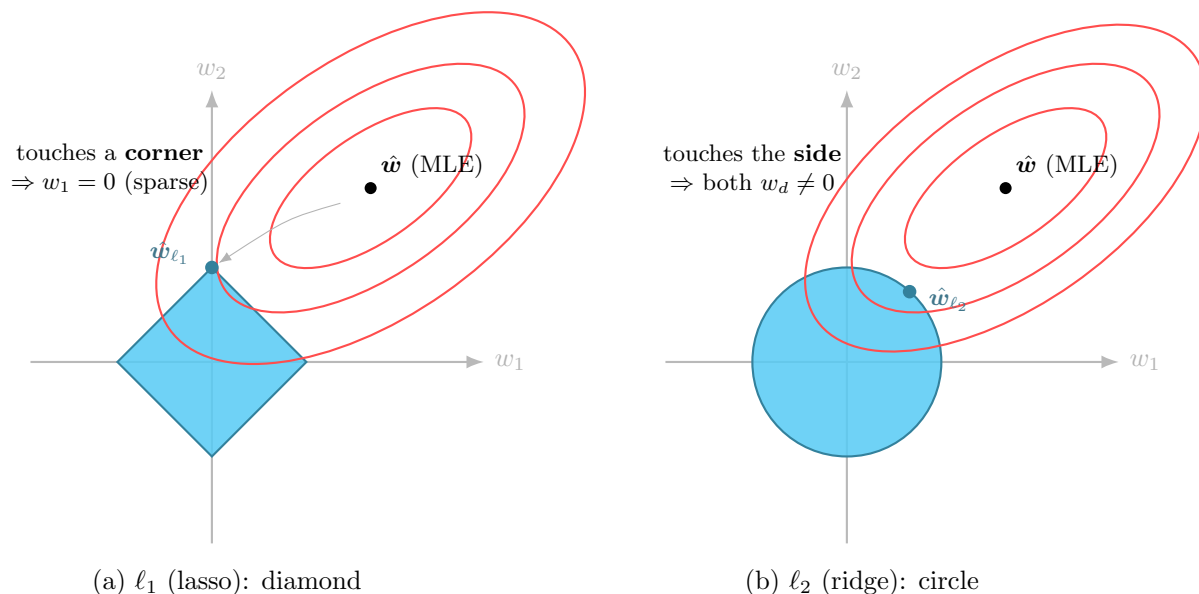
$$\text{PNLL}(\mathbf{w}) = -\log p(\mathcal{D} \mid \mathbf{w}) - \log p(\mathbf{w} \mid \lambda) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda\|\mathbf{w}\|_1, \quad \|\mathbf{w}\|_1 \triangleq \sum_{d=1}^D |w_d|.$$

This is **lasso** — “least absolute shrinkage and selection operator.” More generally, MAP estimation with a Laplace prior is called ℓ_1 **regularization**. (Compare ridge, which used the ℓ_2 penalty $\lambda\|\mathbf{w}\|_2^2$ from a Gaussian prior.)

Why ℓ_1 ? The q -norm and convexity. One could penalize any q -norm,

$$\|\mathbf{w}\|_q = \left(\sum_{d=1}^D |w_d|^q \right)^{1/q},$$

and smaller q gives sparser solutions, with the limit $q = 0$ recovering the ℓ_0 count $\|\mathbf{w}\|_0 = \sum_d \mathbb{I}(|w_d| > 0)$. But for *any* $q < 1$ the penalty is **non-convex** (hard to optimize). The ℓ_1 norm is the *tightest convex relaxation* of ℓ_0 — the sparsest-inducing penalty that still gives a convex, tractable problem.



2.6.2 Why Does ℓ_1 Regularization Yield Sparse Solutions?

The lasso objective is *non-smooth* (the ℓ_1 norm has kinks where any $w_d = 0$):

$$\min_{\mathbf{w}} \text{NLL}(\mathbf{w}) + \lambda \|\mathbf{w}\|_1.$$

This is the Lagrangian form of a bound-constrained problem:

$$\min_{\mathbf{w}} \text{NLL}(\mathbf{w}) \quad \text{s.t.} \quad \|\mathbf{w}\|_1 \leq B,$$

where the bound B trades off against the penalty: a *tight* (small) B corresponds to a *large* λ , and vice versa. Ridge has the analogous form with an ℓ_2 constraint, $\|\mathbf{w}\|_2^2 \leq B$.

Geometric argument. By constrained-optimization theory, the solution sits where the *lowest* level set of the NLL (the innermost RSS ellipse it can reach) first touches the constraint surface (assuming the constraint is active). Picture growing the constraint “ball” from the origin until it meets the ellipse:

- ℓ_1 (**diamond**): its *corners* lie on the coordinate axes and “stick out,” so the ellipse is far more likely to make first contact at a corner than along a flat side. A corner has one (or more) coordinate exactly zero, hence a **sparse** solution. This bias toward corners grows stronger in high dimensions, where corners stick out even more.
- ℓ_2 (**circle**): smooth, with no corners, so first contact can occur *anywhere* on the surface, generically off-axis. There is no preference for zeros, so ridge shrinks but does not select.

2.6.3 Elastic Net (Ridge and Lasso Combined)

Group lasso requires knowing the group structure in advance. When we don’t, but still want highly correlated features treated as an implicit group, we can use the **elastic net** — a hybrid of lasso and ridge that minimizes

$$\mathcal{L}(\mathbf{w}, \lambda_1, \lambda_2) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \lambda_2 \|\mathbf{w}\|_2^2 + \lambda_1 \|\mathbf{w}\|_1.$$

It combines the two penalties: the ℓ_1 term (λ_1) drives sparsity / selection, while the ℓ_2 term (λ_2) adds the ridge-style shrinkage.

Strict convexity. For $\lambda_2 > 0$ the penalty is *strictly convex*, so there is a unique global minimum, even when $\mathbf{X}$ is not full rank (the ℓ_2 term cures the same ill-conditioning ridge did).

Grouping effect. Any strictly convex penalty exhibits a **grouping effect**: the coefficients of highly correlated features are pulled toward each other. In the extreme, if two features are identical ($\mathbf{X}_{:,j} = \mathbf{X}_{:,k}$), then $\hat{w}_j = \hat{w}_k$. Pure lasso lacks this: it tends to arbitrarily pick one of a correlated pair (e.g. $\hat{w}_j = 0$ while $\hat{w}_k \neq 0$), giving less stable estimates.

Beyond the N limit. When $D > N$, lasso can select at most N nonzero variables. Elastic net is not bound by this, it can select more than N nonzeros along its path, exploring more candidate subsets of variables.

2.6.4 Optimization Algorithms

The lasso objective is convex but non-smooth (the ℓ_1 kink), so it needs solvers beyond plain gradient descent. The main families:

Coordinate descent (shooting algorithm). Optimizing all weights at once is hard, but optimizing *one at a time* is easy: holding the rest fixed, solve

$$w_j^* = \underset{\eta}{\operatorname{argmin}} \mathcal{L}(\mathbf{w} + \eta \mathbf{e}_j),$$

where $\mathbf{e}_j$ is the j th unit vector. Cycle through coordinates (deterministically, randomly, or steepest-gradient first). This is especially appealing for lasso because each 1d subproblem has a *closed form* via soft-thresholding — the **shooting algorithm**. It underlies the popular `glmnet` library.

Algorithm 2 Coordinate descent for lasso (shooting algorithm)

```

1: Initialize  $\mathbf{w} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$ 
2: repeat
3:   for  $d = 1, \dots, D$  do
4:      $a_d = \sum_{n=1}^N x_{nd}^2$ 
5:      $c_d = \sum_{n=1}^N x_{nd} (y_n - \mathbf{w}^\top \mathbf{x}_n + w_d x_{nd})$ 
6:      $w_d = \text{SoftThreshold}(c_d/a_d, \lambda/a_d)$ 
7:   end for
8: until converged

```

Projected gradient descent. Make the ℓ_1 penalty smooth via the **split-variable trick**: write $\mathbf{w} = \mathbf{w}^+ - \mathbf{w}^-$ with $\mathbf{w}^+ = \max\{\mathbf{w}, 0\}$, $\mathbf{w}^- = -\min\{\mathbf{w}, 0\}$, so $\|\mathbf{w}\|_1 = \sum_d (w_d^+ + w_d^-)$. This turns lasso into a smooth, *non-negativity-constrained* problem,

$$\min_{\mathbf{w}^+ \geq 0, \mathbf{w}^- \geq 0} \text{NLL}(\mathbf{w}^+ - \mathbf{w}^-) + \lambda \sum_{d=1}^D (w_d^+ + w_d^-),$$

which (for a Gaussian likelihood) is a quadratic program. Solve it by projected gradient descent: take a gradient step, then project onto the positive orthant via $\mathcal{P}_+(w) = \max(w, 0)$ elementwise.

Proximal gradient descent (ISTA / FISTA). Proximal methods handle a smooth loss plus a non-smooth penalty directly. The *proximal operator* of the ℓ_1 penalty is soft-thresholding, so the update is

$$\mathbf{w}_{t+1} = \text{SoftThreshold}(\mathbf{w}_t - \eta_t \nabla \text{NLL}(\mathbf{w}_t), \eta_t \lambda),$$

applied elementwise. This is the **iterative soft-thresholding algorithm (ISTA)**. Adding Nesterov acceleration gives **FISTA**, which is widely used for fitting sparse linear models.

LARS (homotopy / path methods). Some methods compute the entire **regularization path** — all solutions $\hat{\mathbf{w}}(\lambda)$ at once — by exploiting *warm starting*: $\hat{\mathbf{w}}(\lambda_k)$ is cheap to get from $\hat{\mathbf{w}}(\lambda_{k-1})$ when $\lambda_k \approx \lambda_{k-1}$. Even for a single target λ^* , sweeping from $\lambda_{\max}$ down to λ^* (a **continuation / homotopy method**) is often faster than cold-starting, especially for small λ^* .

LARS (“least angle regression and shrinkage”) is the homotopy method for lasso. It starts at large λ (only the feature most correlated with $\mathbf{y}$ is active), then decreases λ until a second feature ties it

in correlation with the current residual $\mathbf{r}_k = \mathbf{y} - \mathbf{X}_{:,F_k} \mathbf{w}_k$ (where F_k is the active set). The next λ at which the active set changes is found *analytically* via a geometric (“least angle”) argument, so the algorithm jumps directly between breakpoints until all variables enter. Allowing variables to also *leave* the active set recovers the exact lasso path; disallowing removals gives **LAR**, which is close to greedy forward selection and least-squares boosting.

	 Ridge (ℓ_2) Gaussian prior	 Lasso (ℓ_1) Laplace prior	 Elastic Net Gaussian $\times$ Laplace
Penalty	$\lambda \ \mathbf{w}\ _2^2$	$\lambda \ \mathbf{w}\ _1$	$\lambda_1 \ \mathbf{w}\ _1 + \lambda_2 \ \mathbf{w}\ _2^2$
Geometry	smooth ball (no corners)	diamond $\rightarrow$ corners on the axes	rounded diamond (corners + curvature)
Effect on weights	shrinks all toward 0, rarely <i>exactly</i> 0	sparsifies : drives many to <i>exactly</i> 0	shrinks <i>and</i> zeros out (both effects)
Feature selection	 no (keeps all D)	 yes (sparse weights)	 yes
Unique solution?	 strictly convex - unique even if $\mathbf{X}$ rank-deficient	convex but <i>not</i> strict, solution may be non-unique	 strictly convex ($\lambda_2 > 0$) - unique
Optimization	smooth; closed form $(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$ (QR/SVD)	non-smooth; coord. descent, ISTA/FISTA, LARS	non-smooth; same solvers as lasso
Correlated features	keeps all, <i>spreads</i> weight evenly (stable)	picks <i>one</i> arbitrarily, drops the rest (unstable)	grouping effect : correlated $\rightarrow$ similar coeffs, in/out together
When $D > N$	keeps all D features	selects at most N	can select <i>more</i> than N
Use when...	multicollinearity; all features relevant; want stability	want a sparse, interpretable model; feature selection	correlated groups + selection; $D \gg N$